

determined to be infeasible.

Planner: making exploitation explicit and revisable. A key challenge in automated exploitation is that naïvely generated commands often fail silently or repeatedly, causing agents to loop without progress. To avoid this failure mode, the Planner maintains an explicit *Exploit Plan* that decomposes exploitation into a sequence of concrete, inspectable steps. Each step specifies a goal, an action, and a status (e.g., *planned*, *done*, or *blocked*), enabling transparent tracking of progress and failures. This explicit plan representation allows the system to reason *about* the exploitation process itself, rather than reacting myopically to the latest output.

At the start of exploitation, the Planner performs a grounding phase that mirrors how human security experts approach an unfamiliar target. It interprets the vulnerability description and evidence chain, retrieves relevant security knowledge from vulnerability documentation, scans the codebase to understand the technology stack and attack surface, and consults long-term memory for previously successful strategies or technical patterns. This grounding ensures that exploitation actions are informed by both program context and security-domain knowledge, rather than blind trial-and-error. Based on this context, the Planner drafts an initial multi-step Exploit Plan when none exists, or incrementally refines the existing Plan in subsequent iterations. Crucially, plan refinement is *explicit and feedback-driven*. After each execution, the Planner updates the status of the attempted step, marks failures as blocked, and inserts corrective actions when necessary (e.g., adjusting file paths, switching payloads, or trying alternative commands). Importantly, the Planner also revisits future planned steps in light of newly observed evidence, modifying or discarding steps whose assumptions are invalidated by execution feedback. This proactive revision prevents the system from blindly following outdated plans or repeatedly executing ineffective actions, enabling adaptive, long-horizon exploitation reasoning.

From the updated plan, the Planner generates a concrete executable action for the next step, expressed as a command or script. Before execution, each proposed action is passed through a *Validation agent*, exposed to the Planner as a tool. This validation step is critical in security settings, where malformed commands, incorrect assumptions, or unsafe actions can derail execution or invalidate results. The Validation agent acts as a safety and consistency gate, checking that actions are well-formed, syntactically sound, aligned with the intended goal, and compatible with the observed system state. Only validated actions are forwarded for execution, while invalid actions are returned for refinement (details in Appendix A.1).

Execution and evaluation: grounding reasoning in reality. Validated actions are executed by the Execution agent within an isolated Docker-based environment, ensuring realistic interaction with the target system while preventing unintended modification of the original codebase. Execution results include structured status signals, raw outputs, and error messages.

These results are analyzed by the Evaluation agent, whose role is to convert *low-level* execution traces into *high-level* reasoning signals. The Evaluation agent determines whether the execution achieved its intended goal, highlights deviations or unexpected behaviors, identifies environment or configuration errors, and produces concrete suggestions for next steps (e.g., retrying with modified inputs or adjusting exploitation strategy). This feedback closes the loop by directly informing the Planner’s next revision.

Finalization and outcomes. The orchestrator monitors the plan–execute–evaluate loop and determines whether the system should `CONTINUE` iterating, declare `SUCCESS` upon successful vulnerability reproduction, or terminate with `FAILURE` when further progress is unlikely. The output of Stage II is the validated exploitation evidence, such as a proof-of-concept payload or exploit trace, demonstrating vulnerability impact. By explicitly modeling exploitation as an iterative, execution-

grounded reasoning process, Stage II enables robust and adaptive vulnerability validation beyond what static or single-shot approaches can achieve.

3.4. Evolution via long-term memory

Human experts do not analyze vulnerabilities in a vacuum; they leverage experience to recognize patterns and avoid pitfalls. CO-REDTEAM emulates this adaptive growth via *long-term memory*, enabling the system to evolve by distilling lessons from past discovery and exploitation trajectories.

A key challenge is that software vulnerability analysis involves *heterogeneous reasoning processes* that differ fundamentally across stages. Vulnerability discovery requires abstract pattern recognition over code structure and data flow, while exploitation demands both high-level strategic planning and low-level technical execution. These reasoning modes operate at different levels of abstraction and generate experience at different granularities. Consequently, a single, homogeneous memory representation is insufficient to support effective reuse across the workflow. Motivated by this observation, we adopt a *layered memory design* that explicitly separates vulnerability patterns, strategic insights, and concrete technical actions, aligning stored experience with the reasoning demands of each stage.

(1) Vulnerability Pattern Memory captures confirmed vulnerability schemas, distilled from validated vulnerability proposals and exploitation outcomes. Each pattern records the progression from observable symptom to vulnerability hypothesis to confirming test, together with common false leads that impeded confirmation. For example, a pattern may encode that a seemingly benign URL fetch function becomes exploitable only when combined with a specific configuration flag, while also recording misleading indicators that initially suggested a different vulnerability class. This memory layer supports rapid recognition of recurring vulnerability structures across codebases.

(2) Strategy Memory captures high-level exploitation strategies abstracted from completed Exploit Plans. These items synthesize generalizable lessons that transfer across targets, such as exploitation workflows applicable to similar vulnerability classes across different applications (e.g., Cross-Site Scripting exploitation strategies across distinct web frameworks). Both successful strategies (e.g., “prioritize configuration analysis before payload crafting”) and failure cases (e.g., “blind fuzzing without understanding the execution context leads to dead ends”) are retained, guiding future planning toward more effective directions.

(3) Technical Action Memory records concrete, low-level actions, such as commands, scripts, or tool invocations, extracted from execution logs and Exploit Plans. When an action succeeds, it is distilled into a reusable “how-to” snippet or technical trick (e.g., a working command for testing SSRF reachability). When an action fails, the associated pitfall and corrective adjustment are stored (e.g., an incorrect file path assumption and its fix). This layer reduces repeated trial-and-error and accelerates execution-level reasoning.

Together, these memory layers allow the system to retain experience at conceptual, strategic, and technical levels, supporting both generalization and precision. Memory items are automatically synthesized using LLM-based extraction over research plans, execution traces, and evaluation feedback, following principles from prior structured reasoning memory work (Ouyang et al., 2025). Signals from the Evaluation agent and the Orchestrator determine whether a trajectory yields a successful practice to preserve or a failure lesson to avoid. Memory retrieval is performed via embedding-based similarity search and exposed as a tool to all major agents, enabling experience-informed reasoning throughout both vulnerability discovery and exploitation and allowing the system to improve progressively over time.

4. Experiments

In this section, we evaluate the effectiveness of CO-REDTEAM on multiple challenging cybersecurity benchmarks. Our results show that CO-REDTEAM significantly outperforms baselines across various LLM backbones (Section 4.1). We further conduct ablation studies to demonstrate the importance of key components of CO-REDTEAM (Section 3), and provide additional case studies in Appendix B. We first describe the experimental setup.

Data. We evaluate on three challenging security benchmarks. Cybench (Zhang et al., 2024) is a CTF-based cybersecurity benchmark designed to evaluate LLM agents’ potential in finding and mitigating security threats. BountyBench (Zhang et al., 2025a) targets real-world offensive and defensive cyber capabilities, where we focus on Detect and Exploit tasks. CyberGym (Wang et al., 2025) provides a large-scale and realistic security benchmark that emphasizes reproducing vulnerabilities by generating executable proof-of-concept (PoC) exploits. More details can be found in Appendix A.5.

Baselines. We compare our framework against a diverse set of baselines, including both single-model approaches and agent-based systems. **Vanilla** models directly receive the target codebase as input and are prompted to generate a solution without explicit tool use, execution feedback, or structured planning, serving as a minimal baseline for raw model capability. **OpenHands** (Team, 2024) is a generic coding agent designed for software engineering tasks, which employs iterative code generation and tool use but is not specialized for cybersecurity workflows. **C-Agent**, provided as a baseline in CyBench, is an agent that explicitly incorporates execution feedback into its reasoning loop to iteratively refine solutions. We further include **VulTrail** (Widyasari et al., 2025), a multi-agent framework that adopts a mock-court paradigm for vulnerability detection through structured debate among LLM agents, and **RepoAudit** (Guo et al., 2025), an autonomous LLM-based agent designed for repository-level code auditing.

Experiment setups. All agents in our framework are instantiated using the same backbone LLM to isolate the impact of system design. Code-browsing tools and the Execution agent operate within isolated Docker containers to ensure safety and reproducibility. Both the vulnerability documentation tool and long-term memory utilize the gemini-embedding-001 model, retrieving the top 3 relevant items per query. To avoid cold-start issues, we initialize the memory with a small set of curated items distilled from established security databases and human expert experience. Memory synthesis is performed using gemini-2.5-pro to balance generation quality and cost.

In Stage I, we allow 3 iterations for refining vulnerability proposals. In Stage II, the exploitation loop is capped at 20 iterations by default; we analyze the impact of this budget in an ablation study. Unless otherwise specified, all agents and tools are enabled in the main experiments, with component-wise contributions examined separately via ablations. For baselines, we follow the configurations reported in their original papers. To ensure fair comparison, CO-REDTEAM and all baselines use the same backbone LLM within each experiment. We present Gemini models in the main result and include multiple model families in Appendix C.

Evaluation. We follow the official evaluation pipelines provided by each benchmark and report *success rates* for vulnerability detection and exploitation as the primary metrics. CyBench and CyberGym consist exclusively of exploitation tasks, while BountyBench includes both detection and exploitation tasks.

4.1. Main results

Main quantitative results. Table 1 reports the main results across CyBench, BountyBench, and CyberGym. Overall, CO-REDTEAM consistently achieves the strongest performance across all

Table 1 | **Main results on CyBench, BountyBench, and CyberGym.** We report success rates across different methods and backbone LLMs, where BountyBench contains both Exploit and Detect tasks. (Since RepoAudit and VulTrail are static analysis methods that lack execution capabilities, and Cybench evaluation relies on execution to capture flags, we omit experiments for these two baselines on Cybench.)

Method	Backbone LLM	Cybench	BountyBench		CyberGym
			(Exploit)	(Detect)	
Vanilla	Gemini-2.5-flash	10.3%	7.5%	0.0%	1.2%
	Gemini-2.5-pro	13.6%	12.5%	0.0%	8.3%
	Gemini-3-pro	18.5%	17.5%	0.0%	12.1%
OpenHands	Gemini-2.5-flash	16.3%	17.5%	0.0%	4.8%
	Gemini-2.5-pro	31.5%	42.5%	0.0%	16.9%
	Gemini-3-pro	45.2%	45.0%	5.0%	20.2%
C-Agent	Gemini-2.5-flash	18.2%	20.0%	0.0%	5.1%
	Gemini-2.5-pro	31.8%	40.0%	2.5%	15.8%
	Gemini-3-pro	47.8%	47.5%	5.0%	21.5%
VulTrail	Gemini-2.5-flash	N/A	0.0%	0.0%	1.4%
	Gemini-2.5-pro	N/A	7.5%	0.0%	3.1%
	Gemini-3-pro	N/A	10.0%	0.0%	5.6%
RepoAudit	Gemini-2.5-flash	N/A	5.0%	0.0%	3.7%
	Gemini-2.5-pro	N/A	15.0%	0.0%	12.4%
	Gemini-3-pro	N/A	25.0%	2.5%	18.3%
CO-REDTEAM	Gemini-2.5-flash	31.8%	32.5%	7.5%	12.1%
	Gemini-2.5-pro	59.1%	60.0%	12.5%	31.5%
	Gemini-3-pro	63.7%	65.0%	20.0%	37.3%

benchmarks and backbone models, demonstrating effectiveness in both vulnerability detection and exploitation. Among the baselines, agent-based methods that incorporate interaction and execution feedback, such as OpenHands and C-Agent, show clear improvements over static approaches. In contrast, VulTrail and RepoAudit exhibit notably weaker performance, particularly on exploitation tasks, as they do not incorporate execution feedback into their reasoning loop. This gap highlights the critical role of interaction with the execution environment and iterative refinement for validating and reproducing real-world vulnerabilities. By tightly integrating planning, execution, and evaluation in a closed-loop manner, CO-REDTEAM substantially outperforms these methods. For example, with Gemini-3-Pro, CO-REDTEAM achieves 63.7% ASR on CyBench, 65.0% exploit success and 20.0% detection accuracy on BountyBench, and 37.3% ASR on CyberGym, representing large absolute gains over the strongest baselines. These results underscore the effectiveness of our security-aware multi-agent design, which combines domain knowledge, code analysis, execution-grounded iteration, and long-term memory to address the full vulnerability lifecycle.

4.2. Ablation studies

Impact of critical components. In Table 2, we analyze the contribution of key components in CO-REDTEAM. Removing execution feedback leads to the most severe performance degradation across all benchmarks (e.g., ASR drops from 59.1% to 17.5% on CyBench), confirming that execution-grounded

Table 2 | **Ablation study.** Impact of removing individual components (critique, validation, vulnerability documents, code browsing, long-term memory, and execution) from CO-REDTEAM.

	Cybench	BountyBench		Cybergym
		(Exploit)	(Detect)	
CO-REDTEAM	59.1%	60.0%	12.5%	31.5%
No Critique	N/A	N/A	10.0% 2.5%↓	N/A
No Validation	52.3% 6.8%↓	42.5% 17.5%↓	7.5% 5.0%↓	28.3% 3.2%↓
No Vul-doc	55.2% 3.9%↓	52.5% 7.5%↓	10.0% 2.5%↓	30.2% 1.3%↓
No Code Browser	47.5% 11.6%↓	42.5% 17.5%↓	7.5% 5.0%↓	27.9% 3.6%↓
No Memory	50.0% 9.1%↓	40.0% 20.0%↓	7.5% 5.0%↓	22.6% 8.9%↓
No Execution	17.5% 41.6%↓	12.5% 47.5%↓	0.0% 12.5%↓	14.3% 17.2%↓

interaction is indispensable for validating and reproducing vulnerabilities. Disabling long-term memory also results in substantial declines, particularly on CyberGym, highlighting the importance of experience reuse for long-horizon exploitation. The absence of code browsing tools or security documentation (vul-doc) consistently degrades performance, demonstrating that effective vulnerability analysis requires both precise code understanding and domain-specific security knowledge. Removing the validation agent significantly harms performance, indicating that sanity checks on planned actions are critical to prevent ineffective or erroneous executions. Finally, disabling the critic agent mainly affects detection performance, suggesting its role in refining and filtering vulnerability hypotheses before exploitation. Overall, the ablation results show that CO-REDTEAM’s strong performance arises from the synergistic integration of planning, execution, validation, critique, and memory, rather than any single component in isolation.

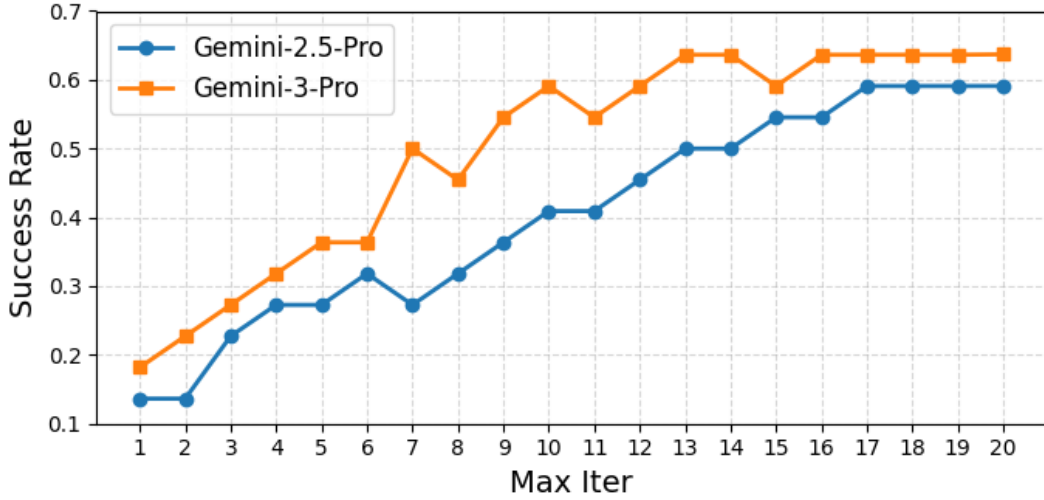


Figure 2 | **Effect of maximum exploitation iterations.** Success rate on CyBench versus maximum exploitation iteration, illustrating how CO-REDTEAM benefits from iterative planning.

Influence of max exploitation iteration. While we set the default maximum iteration of the exploitation loop as 20, we observe that CO-REDTEAM merely uses up all iterations and usually terminates at around 13 to 18 iterations. Therefore, we conduct experiments to investigate the effect